INTERNSHIP PROJECT REPORT

Students Database CRUD Application

Python Backend Development | EWB Edu Tech Pvt. Ltd.

Prepared by	Prince Raj
Email	princerajsinha979@gmail.com
Technology	Python · FastAPI · SQLite
Project type	Individual backend development project

Project focus: reliable student record management through Create, Read, Update and Delete API operations. The local AI extension is optional.

1. Introduction

Thisprojectis a backend REST API for managing student records. It uses Python and FastAPI to expose HTTP endpoints and SQLite to persist records in a local database. The API can create, list, filter, retrieve, update, and delete student records.

2. Objectives

• Implementall fourCRUD operations.

• Store records persistently in SQLite.

• Validate input and handle duplicate email and missing records.

• Provide API documentation and repeatable setup/testing instructions.

• Explore an optional local AI question-answering extension.

3. Technology stack

Technology	Role
Python	Application language
FastAPI	REST API routes and interactive documentation
SQLite	Local relational database
SQLAlchemy	ORM for table models and database operations
Pydantic	Request/response validation
Uvicorn	Development server
Pytest + HTTPX	Automated API testing
Ollama (optional)	Local language model integration

4. Student data model

Field	Description
student_id	Unique database-generated primary key
name	Student name
date_of_birth	Date of birth, YYYY-MM-DD
email	Unique contact email
phone	Optional phone number
course	Optional course
address	Optional address
enrollment_date	Optional enrollment date

5. API endpoint summary

Method	Endpoint	Purpose
POST	/students	Create record
GET	/students	List/filter records
GET	/students/{student_id}	Retrieve one record

Method	Endpoint	Purpose
PUT	/students/{student_id}	Update record
PATCH	/students/{student_id}	Update selected fields
DELETE	/students/{student_id}	Delete record
POST	/ai/ask	Optional AI question

6. Architecture and workflow

• Client sends an HTTP request to a FastAPI route.

• Pydantic validates request fields and data types.

• The route calls a database operation in crud.py.

• SQLAlchemy reads or updates the SQLite database.

• The API returns JSON and an appropriate HTTP status code.

7. Validation and errors

The application validates fields, uses a unique email constraint, returns HTTP 404 when a requested student does not exist, and reports duplicate email as a conflict. Successful deletion returns HTTP 204. Dates should be supplied in YYYY-MM-DD format.

8. Installation and execution

```
python3 -mvenv.venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

On Windows, activate with .venv\Scripts\Activate.ps1. Open http://127.0.0.1:8000/docs to test the endpoints using Swagger UI.

9. Testing plan

• Create a fictional student and check the returned ID.

• List records and retrieve the created record by ID.

• Update a field and verify the new value.

• Delete the record and verify a later GET returns 404.

• Try invalid input and a duplicate email.

• Run pytest -q after installing the project dependencies.

Verification note: Automated tests passed in the build environment (2 tests). Re-run them locally after installing requirements to confirm the same result on your computer.

10. Optional Ollama AI extension

The optional endpoint can answer natural-language questions using student records selected by backend code. The language model is not allowed to execute unrestricted SQL. Ollama and a model such as llama3.2:1b must be installed separately; the core CRUD API does not depend on Ollama.

11. Security and limitations

Use dummy data for demonstrations. Do not upload real student personal information, passwords, API secrets, the local SQLite database, or the virtual environment to GitHub. This is an educational project; production use would

require authentication, authorization, deployment configuration, logging, and further security review.

12. Conclusion

The project demonstrates REST API design, database persistence, CRUD operations, validation, error handling, and automated testing. The optional local AI extension illustrates how a language model can answer questions using records selected by the application.